

App Dev Project Report (MAD 2)

1. Student Details

Name: Mohammad Kashan

Roll Number: 23F2003821

Email: 23f2003821@ds.study.iitm.ac.in

About Me:

I am a Diploma Level student in the BS Degree Program in Data Science and Applications at Indian Institute of Technology Madras. I am highly interested in technology, data science, and building real-world solutions that solve practical workflow problems.

2. Project Details

Project Title: Placement Portal Application (Advanced Backend System)

Problem Statement:

To design and implement a scalable role-based placement management system with REST APIs, background task automation, and asynchronous workflows. The system streamlines campus recruitment by enabling administrators to manage the platform, companies to conduct hiring drives, and students to apply and track placements digitally.

Approach:

The system was developed using Flask with a modular Blueprint architecture. MAD-2 extends the MAD-1 project by converting the system into an API-driven backend with Vue-based frontend integration, background task processing, caching, and automated reporting.

The backend exposes RESTful APIs for all platform operations. The frontend is implemented using Vue.js (CDN version) with modular JavaScript files for each dashboard and workflow. SQLite is used for persistent storage with SQLAlchemy ORM for data modeling.

3. AI / LLM Declaration

AI tools (ChatGPT and Gemini) were used selectively for debugging assistance, conceptual understanding, UI improvement ideas, and documentation formatting.

The overall system design, database modeling, feature implementation, testing, integration, and workflow logic were developed independently.

AI was used as a learning aid to understand issues and implement solutions manually, not for direct copy-paste development.

Extent of AI usage: Approximately 14 –15%.

I am comfortable explaining all parts of the codebase during the viva.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy	ORM for database modeling and interactions
SQLite	Persistent relational database
Flask-Caching(Redis)	API response caching
Flask-Caching(Redis)	Background task processing
Flask-Mail	Email notification
Vue.js (CDN)	Reactive frontend framework
JavaScript	Dashboard logic and API integration
Bootstrap	Responsive UI design and layout

5. Database Schema Design

Tables:

User — Stores authentication and role information

Fields: id (PK), email (Unique), password, role, is_active

Student — Stores student profile data

Fields: id (PK), user_id (FK), name, branch, cgpa, skills, experience, education, resume

Company — Stores company profile data

Fields: id (PK), user_id (FK), company_name, website, hr_contact, approved

JobPosition — Stores placement drive details

Fields: id (PK), company_id (FK), title, salary, skills_required, description, location, experience_required, benefits, eligibility, status, approved, created_at

Application — Bridge table for student job applications

Fields: id (PK), student_id (FK), drive_id (FK), status, interview_date, feedback, applied_on

Constraint: UNIQUE(student_id, drive_id)

Placement — Stores final placement records

Fields: id (PK), student_id (FK), company_id (FK), position, salary, joining_date

Relationships:

One-to-One → User → Student

One-to-One → User → Company

One-to-Many → Company → JobPosition

One-to-Many → Student → Application

One-to-Many → JobPosition → Application

Many-to-Many → Student ↔ JobPosition via Application

One-to-Many → Student → Placement

One-to-Many → Company → Placement

(ER diagram)

6. API Resource Endpoints

Endpoint — Method — Description

/auth/register/student — POST — Student registration
/auth/register/company — POST — Company registration
/auth/login — POST — User authentication
/auth/logout — GET — End session

/admin/dashboard — GET — Platform statistics
/admin/all-jobs — GET — View all jobs
/admin/all-companies — GET — View companies
/admin/approve-company/ — PUT — Approve company
/admin/approve-job/ — PUT — Approve job
/admin/students — GET — View all students

/company/dashboard — GET — Company analytics
/company/create-job — POST — Create job drive
/company/view-jobs — GET — View own jobs
/company/applicants/<job_id> — GET — View applicants
/company/update-application/ — PUT — Update candidate status

/student/dashboard — GET — Student dashboard
/student/jobs — GET — Browse & filter jobs
/student/apply/<job_id> — POST — Apply for job
/student/my-applications — GET — Track applications
/student/update-profile — PUT — Update profile
/student/export — GET — Export applications CSV

7. Architecture and Features

Architecture Overview:

app.py — Flask application factory and configuration
/routes — Modular Flask Blueprints (auth, admin, company, student)
/models — Database models using SQLAlchemy ORM
/tasks — Celery background task modules
/frontend — Vue-based UI with index.html and modular JS files
/static — CSS and static assets
SQLite database — Persistent storage

Core Features:

Role-based authentication (Admin / Company / Student)
REST API driven architecture

Vue.js reactive frontend dashboards
Company approval and job approval workflow
Job posting and application management
Interview scheduling and candidate tracking
Placement finalization system

Advanced MAD-2 Features:

Background email reminders for interviews (Celery)
Automated monthly placement report generation
Asynchronous CSV export processing
Redis caching for performance optimization
Offer letter generation (HTML view)
Resume upload and file serving system

8. Video Presentation

Drive Link:  MAD_2_Project_Presentation.mp4

End of Report